NODIRBEK MAKHTUMOV

AI/ML Engineer Portfolio

Computer Vision · Edge AI · Model Optimization

E-7 Special Ability · Authorized to work in Korea

stark.developer2877@gmail.com · +82 10-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io
Daejeon, South Korea · November 2026

Contents

Employment

01	Smart Waste Collection Fleet Management	2023.04 – Present	2
02	Edge-Aware Segmentation Pipeline	2023.09 – Present	4
03	SAM2 Automated Video Labeling Tool	2024	6

Freelance contracts

04	Smart Truck Parking and Truck-ID Recognition	2025.09 – 2025.11	7
05	Face Recognition Attendance Kiosk	2026.07	10
06	Sales Agent (Email Quoting with Human Approval)	2026.10	12

Personal projects

07	Robustness of a Malware Image Classifier	2026	14
08	IoT Security Gateway (Orange Pi 5 · RK3588S)	2026	16
09	Smart Pointer (Slide Highlighting from Speech)	2026	17

Skills · Education · Languages · Visa 19

Summary

I build computer vision systems that actually run on edge hardware.

- I have spent **3+ years** developing and shipping production vision systems for object detection, segmentation and real-time video analysis.
- I handle the whole path myself, from model design and training through **TensorRT/ONNX quantization and pruning** to deployment on NVIDIA Jetson, Hailo-8 and RK3588 NPUs.
- I own systems **end to end**: FastAPI inference services, PostgreSQL metadata logging and Docker-based edge deployment.
- I don't stop at one accuracy figure: I validate models with **per-class metrics, adversarial robustness and per-hardware benchmarks**.
- I also build **LLM agents with human approval**, and I measure them with evaluation sets.
- I speak Uzbek (native), English, Korean and Russian, and hold an **E-7 Special Ability visa**, so I can start work in Korea right away.

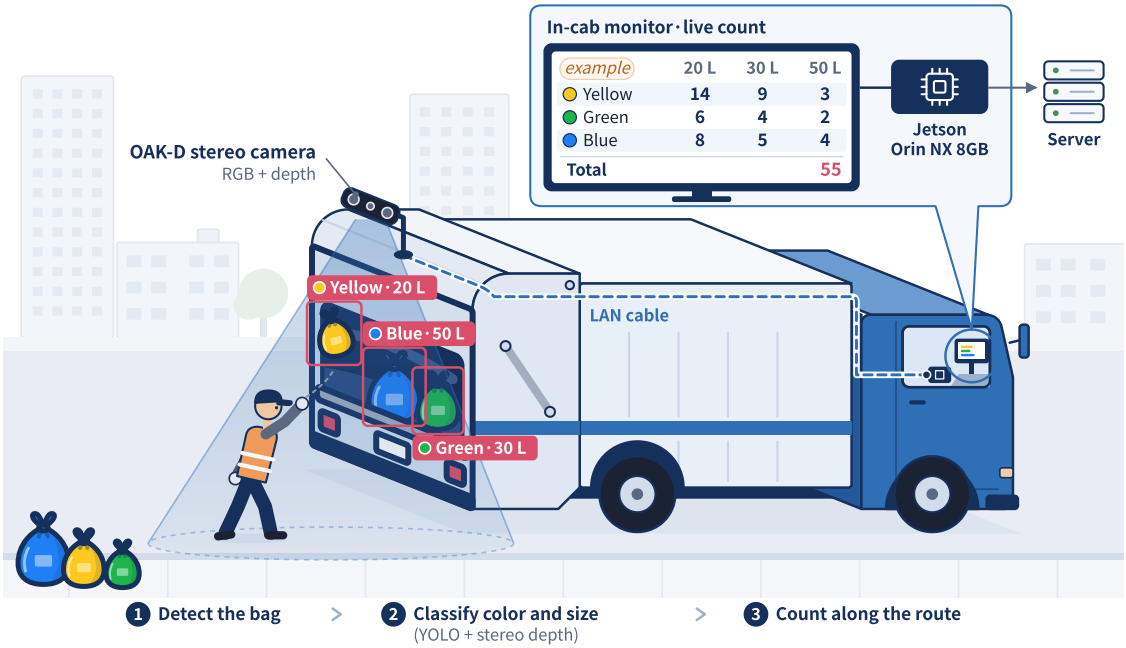
30%	25+ FPS	8.6×	76% → 92%
GPU memory reduction	4 concurrent streams	NPU inference speedup	Truck-ID accuracy
YOLOv8 backbone swap	Real-time segmentation	Hailo-8 · 95× lower jitter	With a deskew module

01 Smart Waste Collection Fleet Management

YOLO waste classification and load estimation — on-vehicle edge inference

Netvision Telecom · AI/ML Engineer · 2023.04 – Present

YOLOv8 | ResNet50 | TensorRT | Jetson Orin NX | DLA | Docker



> **Business impact**
>
> The memory savings let the vision model and the existing fleet-management processes run together, around the clock, on the single Jetson board in each vehicle. The main result was adding vision to the existing hardware without adding boards.

Problem

Waste-type classification and load-volume estimation had to run in real time on a Jetson Orin NX. The stock YOLOv8 model used too much GPU memory to run continuously alongside the other fleet-management processes on the same board.

Technical approach

- **Backbone replacement** — Replaced the default YOLOv8 backbone with ResNet50 and added 1x1 convolution channel adapters to match the feature-map dimensions of the existing PANet neck.
- **DLA offloading** — ResNet-style layers are supported by the Jetson DLA (Deep Learning Accelerator), so I moved that work from the GPU to the DLA. This was the main source of the memory savings.
- **Controlled A/B evaluation** — Compared mAP, FPS and memory use of the original and custom architectures directly on the same target hardware.
- **Optimization and deployment** — Cut runtime further with quantization and TensorRT conversion, and containerized the model with Docker for reproducible deployment within thermal and power limits.

Smart Waste Collection Fleet Management

Architecture comparison

Original architecture

YOLOv8m

Backbone — CSPDarknet53

- **Stem + Conv Layers** — 3 → 64ch
- **C2f Blocks (Stage 2–3)** — 64 → 192ch
- **C2f Blocks (Stage 4)** — 192 → 384ch
- **C2f + SPPF (Stage 5)** — 384 → 576ch

Neck — PANet / FPN unchanged

- **Upsample + Concat + C2f** — Multi-scale fusion

Head — Decoupled unchanged

- **Detect (P3, P4, P5)** — cls + bbox

Modified architecture

YOLOv8 + ResNet50

Backbone — ResNet50 (replaced)

- **Conv7x7 + MaxPool** — 3 → 64ch
- **Layer 1–2 (Bottleneck)** — 64 → 512ch
- **Layer 3 (Bottleneck)** — 512 → 1024ch
- **Layer 4 (Bottleneck)** — 1024 → 2048ch

1x1 Conv channel adapter — added

- **1x1 Conv channel adapter** — 512/1024/2048 → 192/384/576 · fits the neck

DLA offloading — key optimization

- **DLA offloading** — ResNet-style layers (Conv · BN · ReLU) are DLA-supported → moved off the GPU

CSPDarknet's C2f blocks contain custom operations that the DLA does not fully support, so they run only on the GPU.

Neck — PANet / FPN unchanged

- **Upsample + Concat + C2f** — Multi-scale fusion

Head — Decoupled unchanged

- **Detect (P3, P4, P5)** — cls + bbox

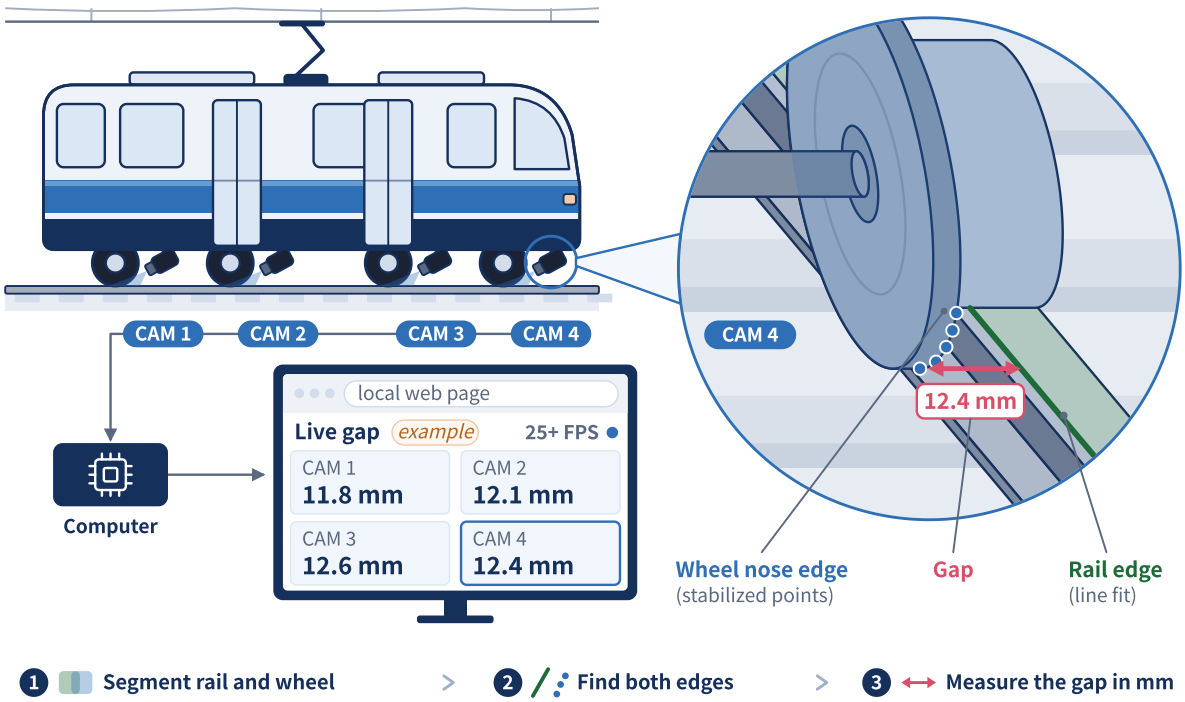
Backbone	Key change	Source of savings	GPU memory
CSPDarknet → ResNet50	1x1 Conv adapter	ResNet layers → DLA offload	↓ 30%

02 Edge-Aware Segmentation Pipeline

Real-time rail-wheel gap measurement — 4 concurrent camera streams

Netvision Telecom · AI/ML Engineer · 2023.09 – Present

PyTorch | RANSAC | FastAPI | PostgreSQL | OpenCV | Multi-threading



1 Segment rail and wheel > 2 Find both edges > 3 Measure the gap in mm

Business impact

Replaced manual rail-wheel gap inspection with automatic measurement from multiple cameras, and stored every measurement with a timestamp so the history can be tracked and analyzed later.

Problem

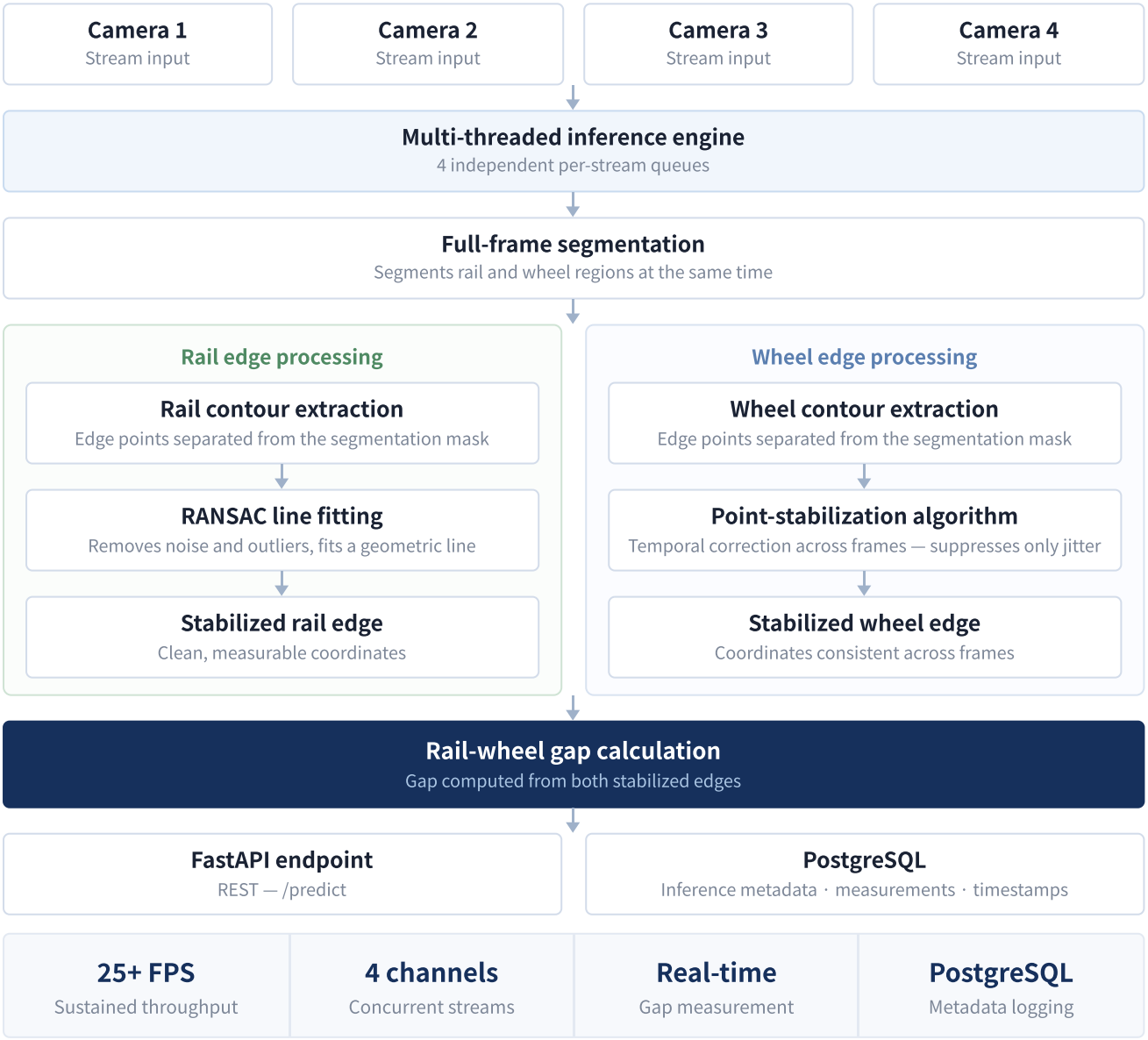
Rail and wheel edges had to be detected precisely, in real time, across several camera streams. Standard segmentation models produced noisy edges, especially on curved boundaries, that shifted from frame to frame, so the gap measurements computed from them could not be trusted.

Edge-Aware Segmentation Pipeline

System architecture

Technical approach

- **Lightweight segmentation model** — A lightweight architecture, designed to keep inference fast without losing edge precision, segments the rail and wheel regions at the same time.
- **RANSAC edge extraction** — Fitted a geometric line to the rail edge with RANSAC regression to remove segmentation noise and outliers.
- **Custom point-stabilization algorithm** — Stabilizes wheel edge points over consecutive frames, suppressing frame-to-frame jitter while preserving real changes in position.
- **Concurrent stream processing** — A multi-threaded pipeline with an independent queue per stream processes 4 camera feeds at once.
- **Serving and metadata logging** — Served through a FastAPI endpoint; inference metadata and gap measurements are written to PostgreSQL in a structured form.



03

SAM2 Automated Video Labeling Tool

An automatic annotation pipeline built on temporal propagation

Netvision Telecom · AI/ML Engineer · 2024

SAM2 | PyTorch | Video Processing | YOLO/COCO Export | OpenCV

> **Business impact**
> Building training datasets was the team's bottleneck. Replacing manual annotation with automatic propagation greatly shortened data preparation for new vision tasks.

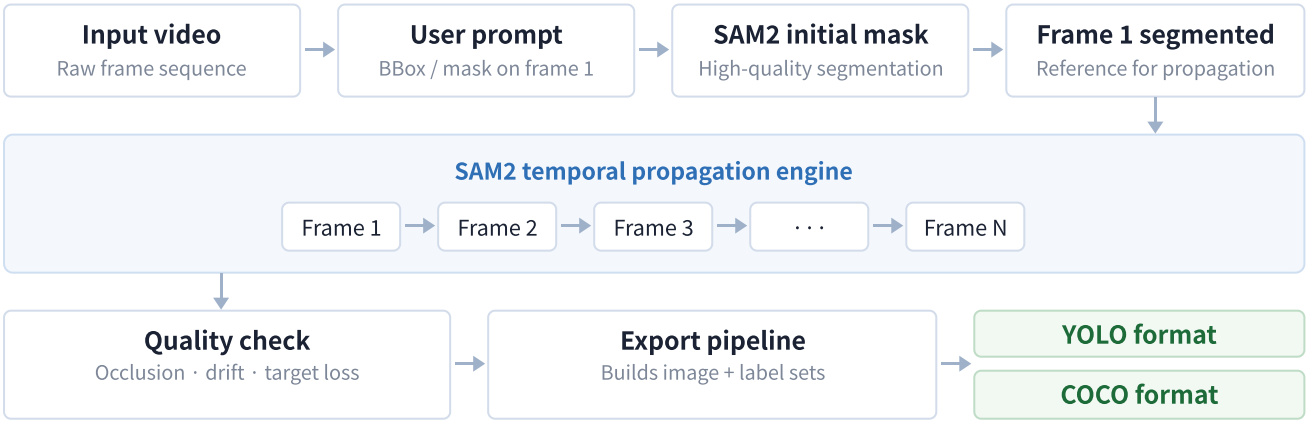
Problem

Training good computer vision models takes large, consistently labeled datasets. Annotating video by hand took far too long, and labeling criteria varied from one annotator to the next.

Technical approach

- **Prompt-based initial segmentation** — The user marks a bounding box or mask once, on the first frame, and SAM2 generates a high-quality initial mask.
- **Temporal propagation** — SAM2's temporal propagation tracks and segments the target through every following frame, labeling the whole video consistently.
- **Label consistency check** — Post-processing filters out propagation failures (occlusion, target loss).
- **Export to training formats** — Labels are converted to YOLO and COCO formats automatically, ready for existing training frameworks.

Pipeline



Results

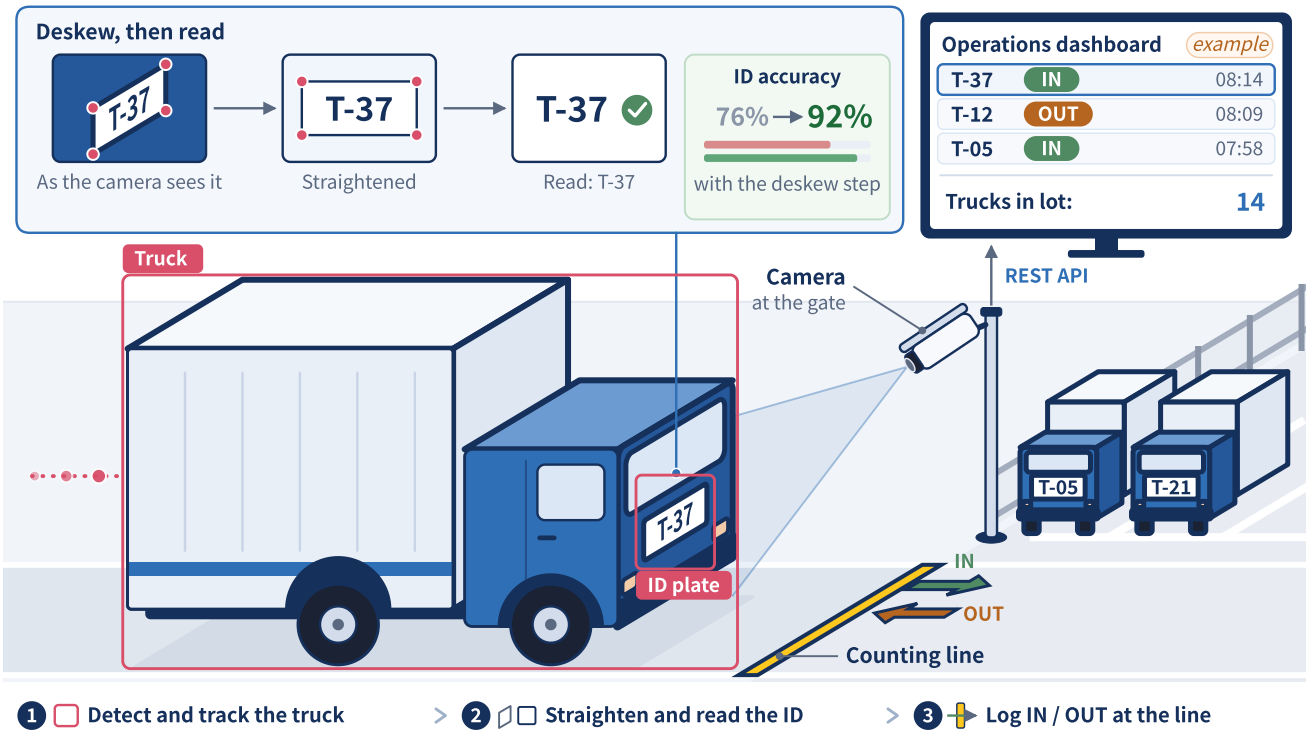
1 frame	Whole video	10x+	YOLO · COCO
Manual input	Labeled automatically	Faster labeling	Export formats

04 Smart Truck Parking and Truck-ID Recognition

A real-time vision system I built end to end, on my own

Freelance contract · AI/ML Engineer · 2025.09 – 2025.11 · **github.com/Nodir0705/smart-truck-parking**

YOLO | ByteTrack | EasyOCR | OpenCV | Perspective Transform | REST API



Business impact

Replaced the handwritten entry log at the truck lot with automatic recognition, and put live occupancy on the operations screen, so operators can see the lot's status without walking around it.

Problem

The truck lot needed automated vehicle detection, unique ID recognition and live occupancy streaming. At the site, ID plates were filmed at oblique angles in poor lighting, and initial OCR accuracy was only 76%.

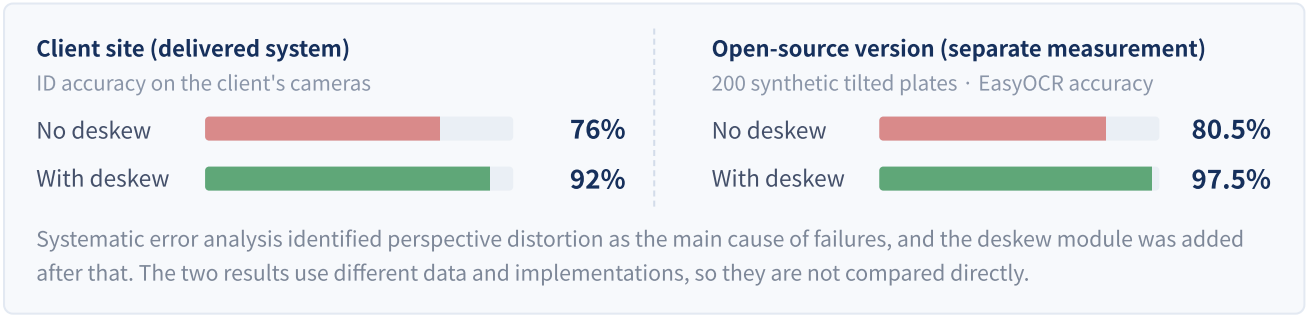
Smart Truck Parking and Truck-ID Recognition

Technical approach

- **Multi-stage detection** — YOLO-based detection finds the truck and the ID-plate ROI separately; only the ID region is cropped and passed on to recognition.
- **Tracking and ID confirmation** — ByteTrack keeps a track for each truck, and line crossing generates IN / OUT events. Each truck ID is confirmed by a vote over the last 5 reads.
- **Custom deskew pipeline** — I wrote a module that finds the plate's four corners with contour detection and adaptive thresholding, then corrects the geometric distortion to a frontal view with a perspective transform and rotation correction.
- **EasyOCR integration** — Tuned to the truck-ID format; contrast enhancement and denoising run first.
- **Real-time streaming** — Entry/exit events and recognized IDs go to the backend over a REST API in real time and appear on the operations dashboard.
- **Systematic error analysis** — Sorting failures by type showed perspective distortion was the main cause, which is why I added the deskew module.
- **Open-source version** — I published an open-source version with a rewritten deskew module. On 200 synthetic tilted plates, EasyOCR accuracy rose from 80.5% to 97.5%. This is a separate measurement from the client-site figure (76% → 92%).

Results

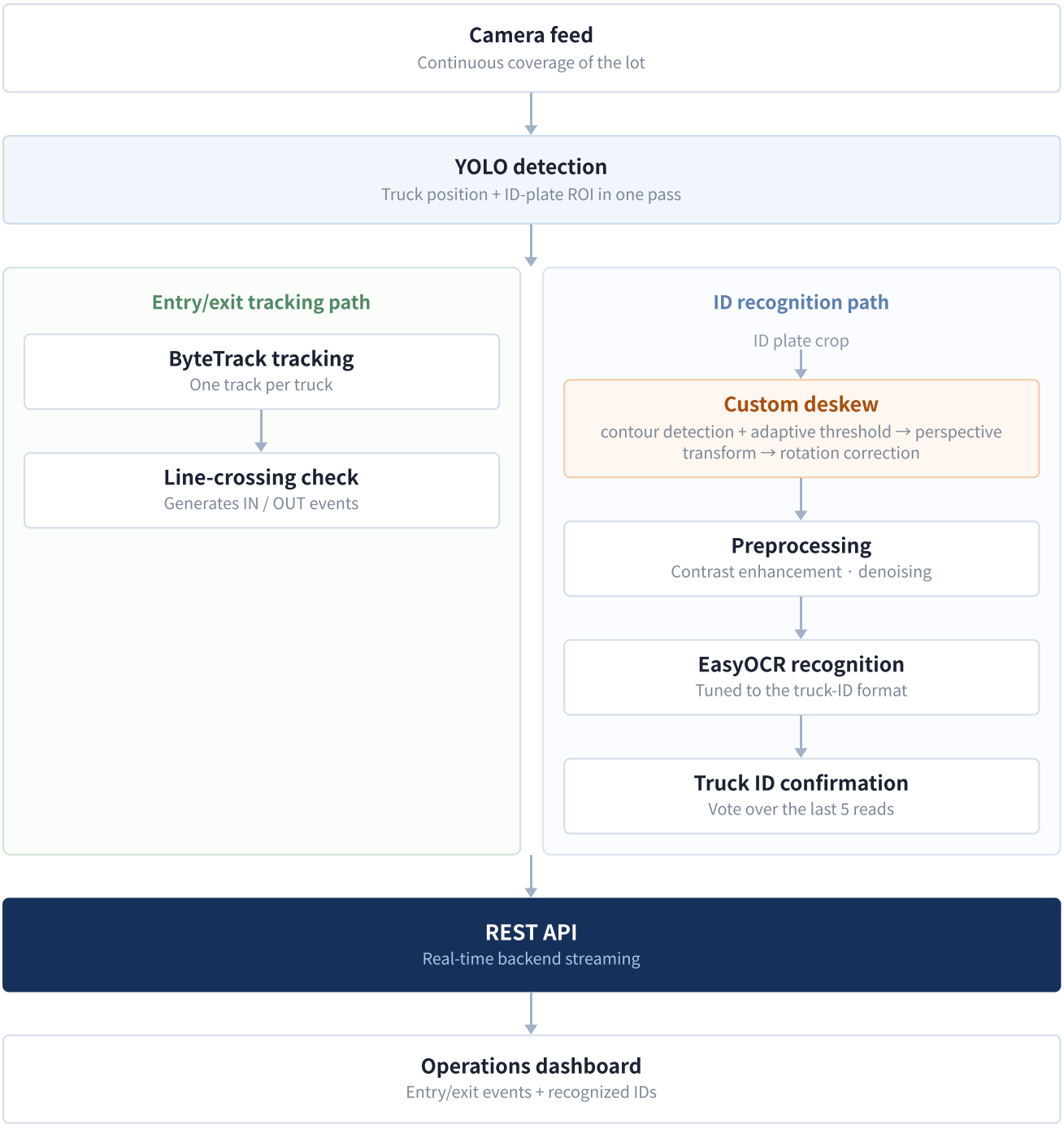
76% → 92%	Real-time	End to end	Custom-built
ID accuracy (client site)	Backend streaming	Built solo	Deskew algorithm



Systematic error analysis identified perspective distortion as the main cause of failures, and the deskew module was added after that. The two results use different data and implementations, so they are not compared directly.

Smart Truck Parking and Truck-ID Recognition

Pipeline



05 Face Recognition Attendance Kiosk

3 pipelines benchmarked, then delivered on-device on a Samsung Galaxy Tab

Freelance contract · AI/ML Engineer · 2026.07 · **github.com/Nodir0705/face-recognition**

SCRFD | ArcFace | InsightFace | Hailo-8 NPU | Raspberry Pi 5 | C++ | Galaxy Tab | TFLite | SQLite

> **Business impact**
> I didn't choose the hardware by guesswork: I built the same pipeline three ways and chose from the measurements. The delivered attendance kiosk runs on-device on a Samsung Galaxy Tab, so a single tablet does the job with no separate inference hardware.

Problem

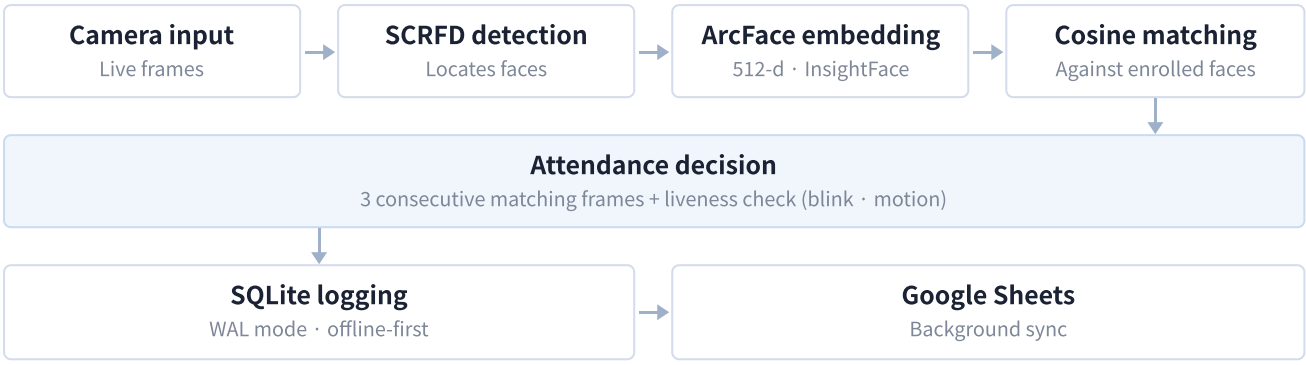
The client needed a real-time face recognition attendance kiosk, but the target hardware had not been decided. CPU inference was slow, but the bigger problem was latency variance. If users cannot predict how long they will wait in front of the camera, the kiosk feels slow.

Technical approach

- **Three builds measured to choose the hardware** — I implemented the same recognition pipeline three ways, Python/CPU, a C++ port and Hailo-8 NPU, and measured each. Python/CPU and Hailo-8 were compared on the same Raspberry Pi 5 with the same image; the C++ port was compared with the Python version on a Raspberry Pi 4.
- **Models** — SCRFD detects faces, and ArcFace 512-d embeddings (InsightFace `buffalo_sc`) are matched by cosine similarity. Attendance is recorded only after 3 consecutive matching frames and a blink and motion liveness check.
- **Guided enrollment wizard** — Modeled on iPhone Face ID: it guides the user through 5 poses (center, left, right, up, down) and captures each pose automatically once it is held steady.
- **Offline-first design** — Records are written to SQLite (WAL mode) first and synced to Google Sheets in the background, so no attendance records are lost when the network drops.
- **On-device delivery on Galaxy Tab** — Based on the measurements, I chose the Samsung Galaxy Tab as the final target and ported inference to run on the tablet itself with Kotlin, CameraX and TFLite. Only one tablet is installed on site.
- **Admin tools** — A Flask web app and an admin dashboard, with tests automated in GitHub Actions.

Face Recognition Attendance Kiosk

Recognition pipeline · measured benchmarks



Python vs Hailo-8 NPU

Raspberry Pi 5 · same image · detection + embedding per face (last column: one frame containing 5 faces)

Pipeline	Median latency	Jitter	Throughput	5-face frame
Python + CPU (4 threads)	48.08 ms	±27 ms	~21 fps	124.21 ms
Hailo-8 NPU	**5.59 ms**	**±0.29 ms**	**~180 fps**	**20.37 ms**

> In practice, the **95× drop in jitter** mattered more than the 8.6× drop in latency. Even with a fast average, a kiosk with high variance feels slow.

Python vs C++ port

Raspberry Pi 4 · one detection + recognition pass (measured separately from the Pi 5 results above)

Implementation	Detection + recognition (1 pass)	Loop overhead
Python	~430 ms	~10 ms
C++ port	~410 ms	~1 ms

> The C++ port only cut loop overhead; total time was dominated by the model itself. Changing the language was not enough. The NPU is what changed the result.

Results

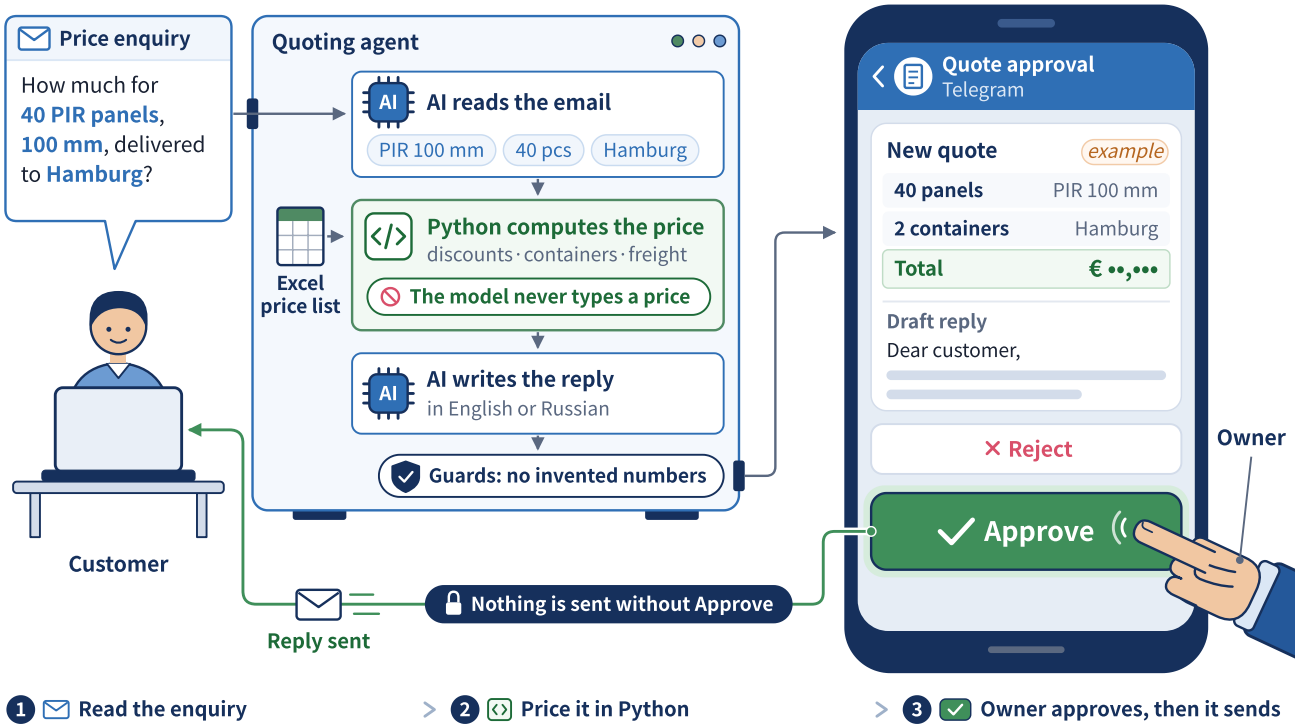
8.6×	95×	3 pipelines	Galaxy Tab
NPU inference speedup	Lower jitter	Measured and compared	On-device delivery

06 Sales Agent

An email quoting agent: Python prices, Claude writes, the owner approves

Freelance contract · AI/ML Engineer · 2026.10 · **github.com/Nodir0705/sales-quote-agent**

Python | Claude API | Structured Outputs | Prompt Caching | Batch API | Telegram Bot API | SQLite



Business impact

The agent does the repetitive part of a price enquiry: it reads the email, works out the quote from the company's price list and writes the reply. The owner reads a card on Telegram and taps Approve or Reject, and nothing reaches a customer before that tap. It has been built and run against a real inbox and Telegram in test mode.

Problem

The client is a company that exports insulated sandwich panels. Most of its sales inbox was the same kind of email: a price for some panels, delivered to a given place. Each one meant opening the Excel price list, applying the discount tier, working out how many containers the order fills, adding freight and writing a polite reply in English or Russian. I built the agent around two rules: the prices come from the price list, not from the model, and the owner sees every reply before it goes out.

Sales Agent

Technical approach

- **Filters before any AI call** — Bounces, auto-replies, newsletters, the inbox's own mail, senders on an ignore list and anyone over the per-sender limit are dropped before a paid call is made.
- **AI reads the email into a fixed schema** — The first Claude call returns structured output: products, quantities, destination, the customer's questions and the language. The email is passed in marked as untrusted data.
- **Python prices it** — Exact decimal arithmetic matches each product and destination in the Excel price list, applies the discount tier, counts containers and adds freight. An ambiguous product or an unknown destination gets a clarification, not a guessed price.
- **The model never types a price** — The second call writes the reply around a placeholder, and Python pastes in its own price block. Python guards then check the draft: a money figure or a time period that Python did not supply blocks sending.
- **Nothing is sent without Approve** — The owner gets a Telegram card with the calculation, the checks and the exact reply. Approval is claimed once in the database, so a double tap or a restart cannot send twice, and the recipient is fixed when the draft is created.
- **Priced calls, no framework** — Every AI call is priced and stored, with a daily cost cap and a per-sender limit; prompts are cached and enquiries that arrive at night go through the Batch API. It is one Python loop and one SQLite file with no agent framework, so the next developer can read it.

Results

26/26	26/26	1,513	Approve
Extraction eval	Drafting test checks	Automated checks	Needed for every reply
Every field correct	Paid API run	All passing	Owner's Telegram card

Written test emails, not client mail · Status: built and run against a real inbox and Telegram in test mode

Test	What it covers	Result
Extraction eval	Written emails in English and Russian: enquiries, follow-ups, questions and unrelated mail; every field must be correct	26/26
Paid drafting test	Real Claude API calls write replies to test emails in English and Russian, including a prompt-injection email	26/26 checks
Automated checks	Pricing, database, guards and Telegram, with every outside service replaced by a fake; run in GitHub Actions	1,513 passing

Limitations: the guards are pattern checks, not a proof (they miss a period with no number in it, such as “a few weeks”), which is why a person approves every reply. Replies are in English or Russian only. The extraction eval uses written test emails, not real client mail.

07

Robustness of a Malware Image Classifier

I built a classifier, then spent more time testing whether to trust its accuracy

Personal project · MalImg dataset (25 families · ~9,000 samples) · 2026 ·
github.com/Nodir0705/malware_classifier

ResNet50 | PyTorch | PGD | Adversarial Robustness | Grad-CAM | MalConv | FastAPI | Docker

> **Business impact**
> This project showed, with numbers, why a model should not be judged on a single accuracy figure. It is where my own checklist for validating production models came from.

Problem

I built a model that classifies malware families by turning each binary, byte by byte, into an image. Test accuracy was 98.95%. The next question was: can this number actually be trusted?

Technical approach

- **Base setup** — I implemented the module that converts each byte of a binary into one grayscale pixel (the public dataset ships only images) and put a 25-way classification head on ResNet50.
- **Class-weighted loss** — Family sizes were imbalanced, from about 3,000 samples down to 80. With class weights, `Autorun.K` went from **F1 0 → 1.0** while overall accuracy barely changed. Looking only at overall accuracy would have missed this.
- **Transfer analysis** — During fine-tuning, the backbone weights moved by **under 1%** (relative L2). The gain came almost entirely from the new classification head, which means ImageNet features transfer to byteplots nearly unchanged.
- **Robustness testing** — Set up reproducible PGD attack and adversarial retraining experiments with a separate data split and a separate baseline model, and compared 3 architectures under an overwrite attack. Results are on the next page.
- **Interpretability and deployment** — Used Grad-CAM to check which parts of a file the model looks at, and packaged the classifier as a FastAPI web app, a CLI and a Docker image.

Results

98.95%	22% → 94.15%	3 models	Grad-CAM
Classification accuracy	Accuracy under PGD attack	Architecture comparison	Interpretability check
Shipped classifier · test set	Separate robustness study	Accuracy vs robustness	Shows where the model looks

Robustness of a Malware Image Classifier

Robustness results

Adversarial attack testing — a reproducible experiment

This experiment is separate from the shipped classifier (98.95%). It is reproducible from the repo, with its own data split (957 test samples) and a separately trained ResNet50 baseline (99.48% clean). Changing each byte by at most 2 (PGD-20, ε=2/255) drops the baseline to 22.15%; by at most 4 (ε=4/255), to 6.58%.

Model	Clean input	PGD-20 ε=2/255	PGD-20 ε=4/255
Baseline (fine-tuned ResNet50)	99.48%	22.15%	6.58%
Adversarially retrained (full network, PGD ε=2/255)	**97.18%**	**94.15%**	**65.20%**

> Retraining the full network on PGD (ε=2/255) samples brought accuracy under attack back to **94.15%**, at a cost of **2.3 points** of clean accuracy. At twice the training strength (ε=4/255), however, it held only 65.20%, so the robustness did not fully carry over to stronger attacks.

Accuracy-robustness trade-off by architecture

Model	Accuracy	Accuracy remaining with 50% of the file overwritten
ResNet50 (image-based)	98.95%	43.50%
MalConv (byte 1D CNN)	94.03%	64.09%
Byte-level Transformer	96.06%	18.27%

> The most accurate model was not the most robust. MalConv has the lowest accuracy but held up best against the overwrite attack, because it focuses on one key byte pattern rather than the whole file.

An unexpected result — few-shot detection of new families

To see whether a new malware family could be recognized from only a few samples, I applied metric learning. The result was **worse, not better**.

	Untrained ImageNet features	Metric-learned version
New-family accuracy (5-way 5-shot)	98.22%	89.40%

> Training pulled related families together in feature space, so a new variant of a known family clustered with its relatives and could not be separated. **Conclusion: untrained ImageNet features are better for detecting new variants.** Not what I hoped for, but without this test I would have picked the wrong design.

Limitation: these figures are on the MalImg dataset, which consists of relatively old Windows malware. Generalization to current malware has not been tested.

08 IoT Security Gateway

From four-layer detection to automatic quarantine on one board, with no cloud

Personal project · Orange Pi 5 (RK3588S) · 2026 · **github.com/Nodir0705/iot-gateway-security**

XDP/eBPF | Suricata | RK3588 NPU | RKNN | ONNX Runtime | Scikit-Learn | nftables | MQTT

> **Business impact**
> A 47-minute stress test confirmed detection-to-quarantine on one board with no cloud.

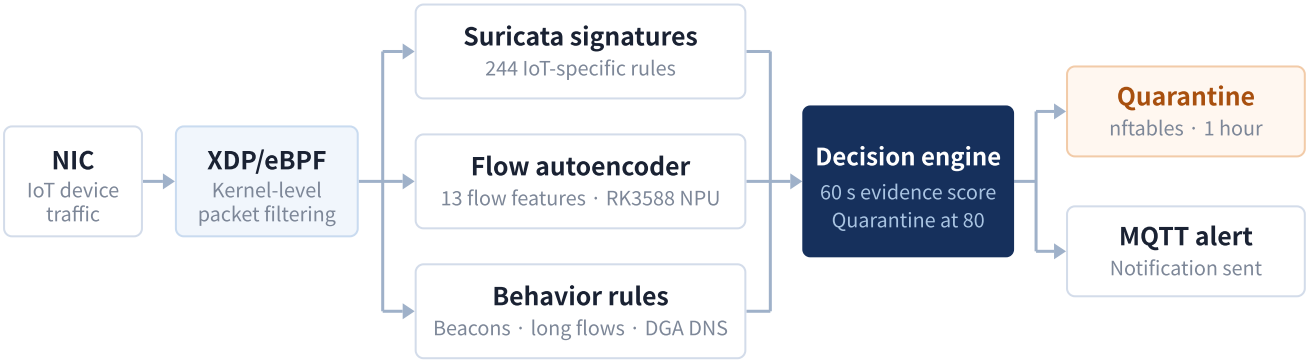
Problem

A single gateway had to watch IoT traffic with no cloud and auto-quarantine any device judged hostile. Covering known attacks and new anomalies meant merging different detection layers into one decision.

Technical approach

- **Kernel-level filtering** — XDP/eBPF filters packets arriving at the NIC first, in the kernel.
- **IoT-specific Suricata rules** — I wrote 244 Suricata rules aimed at IoT attacks.
- **NPU flow autoencoder** — An autoencoder over 13 flow features runs on the RK3588 NPU. Inference falls back from RKNN (NPU) to ONNX (CPU) to a scikit-learn random forest.
- **Behavior rules** — Rules detect beaconing, long-lived connections and DNS queries for DGA domains.
- **Evidence-score quarantine** — Detections from every layer add up to a score. At 80 points within 60 seconds, nftables quarantines the device for 1 hour and an MQTT alert is sent.

Pipeline



Results · 47-minute stress test, 9 attack phases

9/9	87/88 (98.9%)	244	1,551
Attack phases detected	Expected Suricata rules hit	IoT-specific Suricata rules	Alerts logged

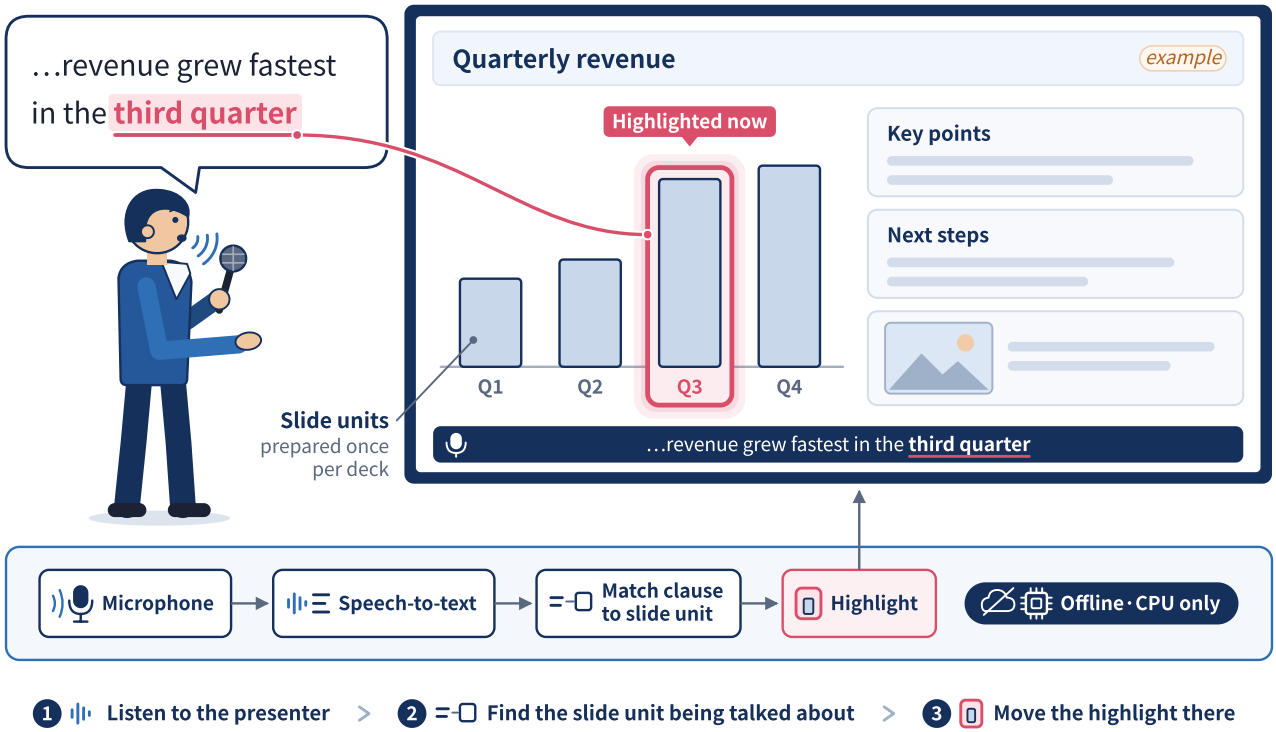
Limitation: lab use only; models trained on NSL-KDD and synthetic flows, not validated on a production network.

09 Smart Pointer

A slide highlight that follows what the presenter is saying

Personal project · 2026 · **github.com/Nodir0705/smart-pointer**

Python | sherpa-onnx | Canary 180M Flash | TinyBERT Cross-Encoder | python-pptx | Claude API



Business impact

While the presenter talks, the part of the slide they are talking about is highlighted, so the audience can see what each sentence refers to. The live loop runs offline on a CPU; the network is needed only once, when a deck is prepared.

Problem

A presenter talks about one part of a slide at a time. I wanted a tool that hears each sentence, works out which box, card, chart bar or table cell it is about and moves the highlight there while the talk goes on. It also has to light nothing when the presenter is not talking about the slide.

Smart Pointer

Technical approach

- **Prepare once per deck** — The .pptx is parsed with python-pptx plus the raw slide XML and rendered; text boxes come exactly from the PDF layer, and OCR reads text in pictures. A model then writes a slide map: the units a presenter would point at, with short phrases they might say about each. The Claude API writes the maps (a local Qwen model is the fallback), so the network is needed once, at prepare time.
- **Live loop on a CPU** — Microphone → speech-to-text (Canary 180M Flash) → clauses → a 2-layer TinyBERT cross-encoder scores each clause against each slide unit → point and hold → highlight in the browser.
- **Point and hold** — The pointer decides once per clause, not on every word. A fixed “none” bar keeps it dark when the presenter is not talking about the slide, and it holds its choice until a new clause clearly points elsewhere.
- **Deterministic** — One CPU thread, pinned model versions and no network in the live loop: the same audio always gives the same highlights, so every change can be compared exactly on recorded talks.
- **Evaluation bench** — Talk scripts read the slides in order, jump around, or refer to things by meaning, and every sentence is labelled with the element it is about. A synthetic (TTS) voice speaks them, and the audio runs through speech-to-text and the pointer faster than real time.
- **Held-out decks** — I downloaded 7 public decks and froze them before testing. Nobody opened their slides during development, and they were scored with the shipped settings, without tuning.

Results

77.3%	2.4	16 ms	2,078
Pointing steps lit	Wrong highlights per talk	Per re-ranker decision	Pointing steps scored
Synthetic voice · 7 decks	Synthetic voice · 21 talks	One CPU thread	On 7 unseen decks

These pointer scores use a synthetic (TTS) voice reading written scripts. No real presentation has been measured yet.

Speech-to-text, chosen by word error rate on a real voice	Word error rate
Canary 180M Flash (chosen)	**11.3%**
Whisper large-v3-turbo	32.1%

Limitations: every pointer score uses a synthetic (TTS) voice, and no real presentation has been measured yet. English only. The slide maps need the network once, when a deck is prepared; only the live loop runs offline.

Technical Stack

Computer Vision	Object Detection, Semantic / Instance Segmentation, OCR, Multi-Object Tracking, Face Recognition (ArcFace), Pose Estimation
ML / DL Frameworks	PyTorch, TensorFlow, Keras, Scikit-Learn, Ultralytics (YOLO), OpenCV, InsightFace, Hugging Face Transformers, SAM2
Model Optimization	Quantization, Pruning, TensorRT, ONNX Runtime, DLA offloading, backbone replacement and A/B evaluation
Edge Hardware	NVIDIA Jetson (Orin NX · Orin Nano), Hailo-8 NPU, RK3588 NPU (Orange Pi 5), Raspberry Pi 4/5, Android tablets
Deployment	Docker, FastAPI, Flask, REST API, PostgreSQL, SQLite, concurrent stream serving
Evaluation · Monitoring	Per-class metrics, adversarial robustness testing, Grad-CAM, FPS/latency/jitter benchmarking, W&B, MLflow, TensorBoard
LLM · Agents	Claude API (structured outputs, prompt caching, Batch API), LLM evaluation, speech-to-text (sherpa-onnx), cross-encoder re-ranking
Languages · Tools	Python (primary), C++, SQL, Linux (Ubuntu), Git/GitHub, GitLab, GitHub Actions CI

Education · Languages · Visa

Education

BSc, Computer Science	Woosong University · Daejeon · 2019 – 2023

Visa

E-7 Special Ability	Authorized to work in South Korea

Languages

Uzbek	Native
English	Fluent (professional working proficiency)
Korean	Conversational
Russian	Conversational

Mentor · Woosong University Capstone Project · 2024

Mentored a computer vision team weekly (image processing, YOLO training); it placed 1st of 200+ teams

Nodirbek Makhtumov

AI/ML Engineer · Computer Vision · Edge AI · Model Optimization

stark.developer2877@gmail.com · +82 10-5751-0750
github.com/Nodir0705 · linkedin.com/in/nodirbek-makhtumov-03084a194
nodir0705.github.io